CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

College of Computer Studies

Bachelor of Science in Information Technology

Software Design Description (SDD v2.0)

PlayIT

An Offline-First Gamified Early Literacy Mobile Application

Course	IT411 — Capstone & Research 2 | Semester 1, AY 2026–2027
Document Version	2.0 (Post-MVP Architectural Baseline)
Publication Date	September 26, 2026 (Revised October 11, 2026)
Document Status	Submitted for Adviser Review and Evaluation
Prepared by	Group 56 — PlayIT Capstone Team: Riva, Zendrix (Team Lead); Palis, John James; Miel, Kaysean; Durano, Ashlee Skye; Bien, Erik Samuel
Adviser	Mr. Joemarie C. Amparo

Table of Contents

Document Revision History 4
1. Introduction 5
1.1 Purpose 5
1.2 System Scope 5
1.3 Definitions and Acronyms 5
1.4 References 6
2. Architectural Design 6
2.0 Implementation Status (checked against the code on 2026-10-09, branch `refactor/hear-say-it`) 6
2.1 System Architecture Paradigm 8
2.2 Decomposition & Architectural Layers 8
3. Detailed Component Design 9
3.1 Audio Subsystem Refactoring (FR-02, NFR-AUD-01) 9
3.1.1 AudioPlaybackManager (planned) 9
3.1.2 AudioComposer Component (planned) 10
3.2 Speech Recognition Subsystem & Visualizer (FR-03, NFR-ASR-01, NFR-PERF-01) 10
3.2.1 AudioRecord Loop & RMS Amplitude Pipeline 10
3.2.2 MicStateVisualizer Composable (planned) 11
3.3 Tutoring Layer & State Machine (FR-03, NFR-ASR-01, FR-NEW-REC) 11
3.3.1 SayItJudge with Dynamic Letter Grammars 11
3.3.2 TutorPolicy Finite State Machine (FSM) 12
3.4 Decodable Blend It Word Bank Refactoring (FR-13) 12
3.5 Database Architecture (Room Schema v4, planned) & Telemetry Logger (FR-14, FR-NEW-TEL) 13
3.5.1 Room Database Entities 13
3.5.2 Local CSV Export Manager 14
3.5.3 System Entity-Relationship Diagram 15
3.6 Pediatric Design System Tokens & Accessibility (NFR-ACC-01, NFR-ACC-02) 16
3.6.1 Touch Targets and Adaptive Dimensions (implemented) 16
3.6.2 Articulation Cue and Sound Captions (implemented; pictures pending) 16
3.7 Module Class Diagrams 16
3.7.1 Module 1: Letter-Sound Acquisition (Hear It) 17
3.7.2 Module 2: Speech Production and Formative Tutoring (Say It) 18
3.7.3 Module 3: Phoneme Discrimination (Find It) 19
3.7.4 Module 4: Word Construction Checkpoint (Blend It) 20
3.7.5 Module 5: Mastery Progression, Gamification and Multi-Profile 21

3.7.6 Module 6: Telemetry, Parent Dashboard and Diagnostics 22
4. Detailed Interface & Interaction Workflows 23
4.1 Hear It Modeling Sequence Workflow 23
4.2 Say It Interaction & Correction Workflow 25
4.3 Find It Tap, Hearts and Restart Workflow 27
4.4 Blend It Word and Group Completion Workflow 28
4.5 Profile, Stars, Streak and Unlock Workflow 29
4.6 Parent Zone, Dashboard and Report Workflow 30
5. Technology Stack & External Dependencies 31
6. Verification & Architectural Testing Plan 31

Document Revision History

Version	Date	Primary Author(s)	Architectural Refactoring & Traceability Description
0.1	May 11, 2026	System Architect	Initial SDD draft based on SRS v2.0 (MVVM, Clean Architecture, Room SQLite, Vosk ASR).
0.2	May 15, 2026	System Architect	Added Word Challenge (Blend It) CVC synthesis checkpoint per adviser directive.
1.0	May 20, 2026	System Architect & Dev Team	Final Capstone 1 SDD: decoupled game modules, established baseline Room DB schema v1.
2.0	September 26, 2026	Lead Architect & Capstone Team	**Comprehensive Renewal & Architectural Refactoring Based on MVP Validation (design; see §2.0 for what is implemented):** • **Tutoring & Pedagogy Layer (§2.2):** Designs `LessonEngine`, `TutorPolicy` finite state machine (FSM), and `SayItJudge` with per-letter dynamic grammars and error tagging; **eliminated heart deductions in Say It**. • **Audio Subsystem Architecture (§3.1):** Refactored `AudioPlaybackManager` with `AudioComposer` and pre-cached `SoundPool` for instant phoneme playback; isolated pure phoneme (`ph_m.wav`) and key-word (`kw_m_mouse.wav`) assets (FR-02, NFR-AUD-01). • **Speech Recognition Subsystem (§3.2):** Plans to replace `SpeechService` (still used today) with an asynchronous `AudioRecord` 16kHz mono loop streaming raw PCM buffers to calculate normalized RMS amplitude for `MicStateVisualizer` while feeding Vosk `Recognizer.acceptWaveForm()`; guaranteed tap-to-listening transition in ≤100ms (FR-03, NFR-PERF-01). • **Decodable Word Bank Refactoring (§3.4):** Purged 5 invalid CVC words (AIM, BEE, TOY, BOY, ZOO) containing untaught vowel teams/diphthongs; replaced with AM, SUM, TUB, YAM, ZIP; flagged QUIZ exception (FR-13). • **Persistence & Telemetry (Room Schema v4, planned, §3.5):** Extends the existing `ProfileEntity` (multi-profile up to 6) and plans `LetterProgressEntity` (spaced retrieval mastery), and `TelemetryEventEntity` (microsecond-accurate monotonic timestamps); added local PIN-gated CSV/PDF exporters (FR-14, FR-NEW-TEL). • **Pediatric Tokens & Accessibility (§3.6):** Plans `Modifier.pediatricTouchTarget(64.dp)` and an `ArticulationCue` composable (NFR-ACC-01, NFR-ACC-02).
2.1	October 5, 2026	Capstone Team (Claude review)	Corrections for adviser review: new §2.0 implementation-status table (implemented vs planned); §3.3.1 judge uses word mode as in the code, with no fixed confidence threshold; Room schema v4 (v3 is current) with explicit migrations; revision 2.0 entries reworded as design, not completed work.
2.2	October 9, 2026	Capstone Team (Claude, sprint synchronization)	§2.0 re-checked against branch `refactor/hear-say-it` after the Oct 9–10 sprint (cards 18–24, 03b, 15). Newly **Implemented**: adaptive dimensions and `LessonScaffold`; 4-state Say It mic (`MicStatus`, `MicButton`, time-based ripple); spoken Say It corrections; decodable Blend It word list; purposeful effects and screen transitions; responsive map (`MapLayout`); sound captions and `ArticulationCue`. §3.4 chapter letters corrected to the seeded groups; §3.6 replaced the design sketches with the implemented components. `LessonEngine`, `AudioComposer`, Room schema v4, telemetry and CSV export remain **Planned**.

Version	Date	Primary Author(s)	Architectural Refactoring & Traceability Description
2.3	October 10, 2026	Capstone Team (Claude, submission review)	Review for the IT411 submission, no design change: §3.1.1 and §3.1.2 use the shipped `assets/audio/` layout and the dated release manifests; §2.2 notes that the planned `tutoring/` classes live in `domain/manager/` today; §3.1, §3.2 and §3.2.2 label the planned components; §4.2 uses the shipped Heard mic state (user decision 2026-10-06); §5 lists the short-vowel source; diagram alignment and §3.5.1 comment wrapping fixed.
2.4	October 10, 2026	Capstone Team (Claude, card 28)	Phone-test refinements (user decisions 2026-10-10): §2.0 adds the card 28 row (Hear It ends on the sound, Say It mic `DONE` after the third miss, Blend It sound-out that waits for each clip, live star totals from one SQL aggregate, one-tap map start) and marks captions as kept in code but not shown; §3.6.2 and §4.1 step 7 follow.
2.5	October 11, 2026	Capstone Team (Claude, diagrams)	Design diagrams added, traced from the code: §3.5.3 system ERD (Figure 1); §3.7 one class diagram per module (Figures 2–7); §4.1–§4.6 one sequence diagram per module (Figures 8–13); planned components are grey and marked «planned». §4.2 now marks `NEEDS_PRACTICE` as planned (Room v4). No design change.

1. Introduction

1.1 Purpose

This Software Design Description (SDD v2.0) provides the definitive technical architecture, component breakdown, data schemas, and interface workflows for **PlayIT**. It translates the refactored functional and non-functional requirements defined in **SRS v3.0** into a concrete, testable, and robust implementation blueprint. This renewed version directly resolves the technical and pedagogical bottlenecks identified during the Weeks 1–2 MVP Field Validation (schwa audio intrusion, microphone button hesitation, missing multi-profile telemetry, non-decodable words, and accessibility targets).

1.2 System Scope

This design document governs the entire PlayIT Android mobile application:

- Three-layer Clean Architecture with Model-View-ViewModel (MVVM) in Jetpack Compose.
- Autonomous, teacher-independent **Tutoring & Pedagogy Engine** (`LessonEngine`, `TutorPolicy` FSM, `SayItJudge`).
- Low-latency Audio Subsystem utilizing pre-cached `SoundPool` and runtime `AudioComposer`.
- Privacy-preserving, completely offline speech recognition via Vosk 0.3.47.
- Room SQLite Database (Schema v4, planned; the app is at v3 today) supporting multi-profile isolation and structured interaction telemetry.
- Pediatric-first design system strictly enforcing a zero-emoji policy and ≥ 64 dp touch targets.
- Local, unauthenticated Parent/Facilitator Dashboard with arithmetic safety gates and PDF/CSV report generation.

1.3 Definitions and Acronyms

- **ASR:** Automatic Speech Recognition.
- **CVC:** Consonant-Vowel-Consonant syllable/word structure.

- **DAO:** Data Access Object (Room persistence layer).
- **FSM:** Finite State Machine.
- **MVVM:** Model-View-ViewModel architectural pattern.
- **PCM:** Pulse-Code Modulation (uncompressed raw audio format).
- **RMS:** Root Mean Square (audio signal power measurement).
- **Vosk:** Lightweight offline speech recognition toolkit.

1.4 References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
3. IEEE Std 1016-2009: *IEEE Standard for Information Technology—Systems Design—Software Design Descriptions*.
4. PlayIT Software Requirements Specification (SRS v3.0), September 2026.

2. Architectural Design

2.0 Implementation Status (checked against the code on 2026-10-09, branch `refactor/hear-say-it`)

This document describes the target design for Weeks 4–9. The table separates what the app does today from what is planned, so the design is not read as a description of the current build.

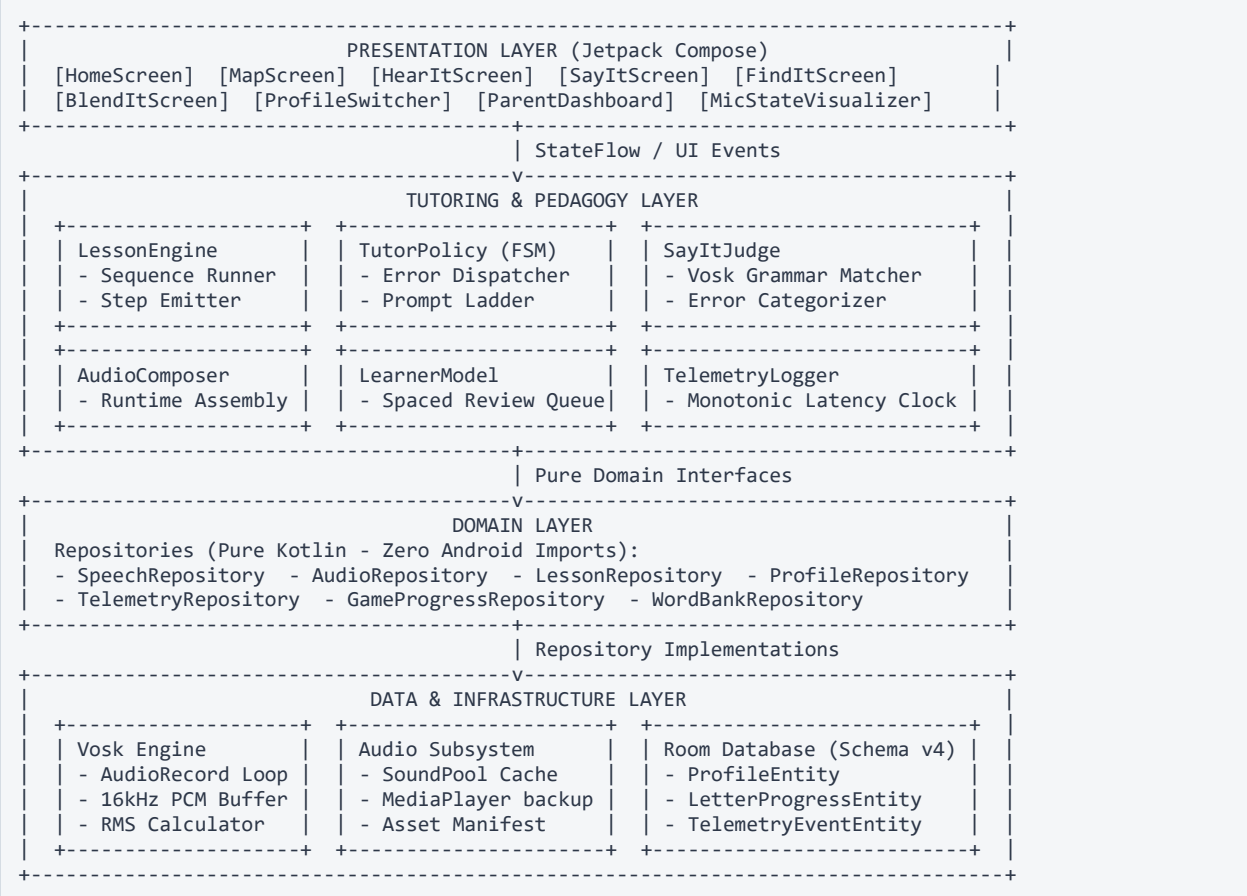
Component	Status	Where / note
`TutorPolicy` (prompt ladder, no hearts in Say It)	Implemented, as a stateless policy	`domain/manager/TutorPolicy.kt`; the FSM in §3.3.2 is the planned form
`SpeechValidator` (word-mode judge, per-letter foil grammars, error types)	Implemented	`domain/manager/SpeechValidator.kt`; plays the role of the planned `SayItJudge`
`VoskRecognizer` with Vosk `SpeechService`	Implemented	`data/speech/VoskRecognizer.kt`; the `AudioRecord` loop of §3.2 is planned (`AudioCapture.kt` exists but is not wired in)
`HearItSequenceBuilder` (modeling sequence)	Implemented	`domain/manager/HearItSequenceBuilder.kt`; a general `AudioComposer` is planned
Idle re-prompt (10 s) and next-step cues	Implemented	`presentation/components/IdleRePrompt.kt`
`HeartManager`, `StarCalculator`, `GridGenerator`, `ArithmeticGateManager`	Implemented	`domain/manager/`
`ProfileEntity`, `SessionManager` (up to 6 profiles)	Implemented	Room schema version 3 (`PlayItDatabase.kt`)
Parent PDF report	Implemented	`data/pdf/PdfExporter.kt`
Adaptive dimensions and lesson layout (`WindowProfile`, `PlayItDimens`, `LessonScaffold`)	Implemented	`presentation/theme/Dimens.kt`, `presentation/components/LessonScaffold.kt`; 64 dp touch and 16 sp text floors; tested in `DimensTest`, `GummyContainerLayoutTest`, `LayoutMatrixTest` (4 device sizes, font scale 1.3)

Component	Status	Where / note
Say It mic states (Idle, Listening, Heard, Result, and Done after the third miss)	Implemented	`presentation/sayit/MicStatus.kt`, `presentation/sayit/components/MicButton.kt`; time-based ripple, no red; returns to Idle when the app is backgrounded; Done takes no taps (card 28); tested in `MicStatusTest`, `SayItViewModelTest`
Spoken Say It corrections (letter name, added vowel, remodel)	Implemented	`presentation/sayit/SayItViewModel.kt`; tutor fragments `fb_letter_name`, `fb_its_sound_is`, `fb_almost_just`, `fb_no_ah`, `fb_listen` (Kokoro, audio release 2026-10-01; teacher audit pending); tested in `SayItViewModelTest`, `AudioResolverTest`
Decodable Blend It word list (§3.4)	Implemented	`di/DatabaseModule.kt` `BLEND_IT_WORD_SEEDS`, written on every open; tested in `BlendItWordSeedsTest`. Teacher confirmation and the AM, TUB, YAM, ZIP word audio and pictures are pending
Purposeful effects and screen transitions	Implemented	`presentation/components/FeedbackEffects.kt`, `navigation/NavGraph.kt`, `CelebrationOverlay.kt`; reduced motion gives fades only; no haptics; tested in `FeedbackEffectsTest`, `PlayItMotionTest`
Responsive map trail, compact header, unlock moment	Implemented	`presentation/map/MapLayout.kt`, `MapScreen.kt`, `MapViewModel.newlyUnlockedNodeId`; tested in `MapLayoutTest`, `TopStatsBarLayoutTest`, `MapViewModelTest`
Mouth-shape cue `ArticulationCue` (§3.6.2); sound captions kept in code, not shown in the child view since card 28 (user decision 2026-10-10)	Implemented	`domain/model/ArticulationGroup.kt`, `domain/manager/CaptionText.kt`, `presentation/components/CaptionBubble.kt`, `ArticulationCue.kt`; mouth pictures await their image release (card 25), until then the cue draws nothing; tested in `ArticulationGroupTest`, `CaptionTextTest`, `ArticulationCueTest`
Phone-test refinements (card 28): Hear It ends on the key word and the sound; the third Say It miss rests the mic and unlocks Next; Blend It sounds out each letter to the end (at least 750 ms) before the whole word; one-tap start of the active map node	Implemented	`domain/manager/HearItSequenceBuilder.kt`, `presentation/sayit/MicStatus.kt`, `presentation/blendit/BlendItViewModel.kt`, `presentation/map/MapTapRules.kt`; tested in `HearItSequenceBuilderTest`, `SayItViewModelTest`, `BlendItViewModelTest`, `MapTapRulesTest`
Live star totals on the map, the profile cards and the parent report	Implemented	`data/local/dao/ProfileDao.kt` sums `lesson_progress` and `blend_it_progress` in SQL, the same sum as `ReportGenerator`; no schema change (card 28); tested in `ProfileDaoStarsTest` on an in-memory Room database
`LessonEngine`, `LearnerModel` (review scheduler)	Planned	—
`AudioComposer`, `AudioPlaybackManager`	Planned	Playback today goes through `data/audio/AudioPlayer.kt`, which already uses a `SoundPool` for short clips
RMS-driven mic ripple (`MicStateVisualizer` with the `AudioRecord` loop of §3.2)	Planned	The 4 mic states are implemented (row above); the voice-driven ripple follows the planned `AudioRecord` loop (post-Round-2, user decision 2026-10-06)

Component	Status	Where / note
`LetterProgressEntity`, `TelemetryEventEntity`, `TelemetryLogger`, `CsvExportManager`	Planned	Needs Room schema v4 and a migration
`Modifier.pediatricTouchTarget()` as a single global modifier	Planned	The 64 dp floor is enforced today through `PlayItDimens` tokens and `heightIn(min = 64.dp)` on every child-facing control

2.1 System Architecture Paradigm

PlayIT is structured around **Clean Architecture** principles combined with **MVVM** and unidirectional data flow (UDF) using Kotlin Coroutines and Jetpack Compose `StateFlow`. To guarantee autonomous child operation without requiring adult supervision, this refactored design situates a specialized **Tutoring Layer** between the Domain and Presentation layers.



2.2 Decomposition & Architectural Layers

1. **Presentation Layer (`presentation/`):** Contains UI composables and ViewModels. ViewModels observe domain StateFlows and emit immutable UI state objects. Composables remain purely declarative and react to state mutations without containing gameplay logic.

2. **Tutoring & Pedagogy Layer (`tutoring/`, planned package):** Governs learner pacing, scaffolding, and formative remediation. Decouples educational decision-making from UI view code. Today `TutorPolicy` and `SpeechValidator` (in the role of `SayItJudge`) live in `domain/manager/` (§2.0):
   - `LessonEngine`: Executes letter lesson scripts in sequential steps.
   - `TutorPolicy`: Implements a finite state machine managing We Do, You Do, error corrections, and lead steps.
   - `SayItJudge`: Evaluates captured speech buffers against constrained per-letter grammars, returning decisions and diagnostic error codes.
   - `LearnerModel`: Tracks memory retention across sessions to power spaced retrieval warm-ups.
3. **Domain Layer (`domain/`):** Houses pure Kotlin enterprise business rules, data models, and repository interfaces. Contains zero `android.*` imports.
4. **Data Layer (`data/`):** Implements domain interfaces using hardware drivers (Vosk ASR, Android AudioRecord, SoundPool) and SQLite persistence (Room 2.6+).

3. Detailed Component Design

3.1 Audio Subsystem Refactoring (FR-02, NFR-AUD-01)

To ensure pure phoneme delivery with zero trailing vowel intrusion and instantaneous acoustic response, the audio subsystem decouples short acoustic phoneme bursts from long musical playback.

3.1.1 AudioPlaybackManager (planned)

- **Underlying Driver:** Android native `SoundPool` API with `AudioAttributes.USAGE_GAME`.
- **Preloading Lifecycle:** When the learner navigates to a chapter node on the Map Screen, all phoneme bursts for that letter (`ph_<letter>.wav`) are loaded into uncompressed memory.
- **Physical Asset Segregation:**
  - `assets/audio/phonemes/ph_<letter>.wav`: Pure phoneme burst (800 ms continuous; ≤ 250 ms stops). Zero schwa trailing (released so far: `ph_m.wav`, `ph_s.wav`).
  - `assets/audio/keywords/kw_<word>.wav`: Key word pronunciation (e.g., `kw_mouse.wav`).
  - `assets/audio/vo/tutor/car_<phrase>.wav`: Spoken carrier phrases (e.g., `car_listen.wav`, `car_this_letter_says.wav`, `car_say_it_with_me.wav`, `car_your_turn.wav`).

3.1.2 AudioComposer Component (planned)

`AudioComposer` will manage runtime assembly of the modeling sequence using duration metadata from the audio release manifests (`docs/audio-release/<date>/manifest.json`); today `HearItSequenceBuilder` builds the sequence (§2.0):

```
class AudioComposer @Inject constructor(
    private val playbackManager: AudioPlaybackManager,
    private val assetManifest: AssetManifestRepository
) {
    suspend fun executeHearItSequence(letter: String, onStepUpdate: (SequenceStep) -> Unit) {
        val script = assetManifest.getScript(letter)
        for (step in script.steps) {
            when (step) {
                is Step.PlayCarrier -> {
                    onStepUpdate(SequenceStep.Carrier(step.id))
                    playbackManager.play(step.id)
                    delay(step.durationMs)
                }
                is Step.RevealMnemonic -> {
                    onStepUpdate(SequenceStep.Mnemonic(letter))
                    delay(step.durationMs)
                }
                is Step.PlayPhoneme -> {
                    onStepUpdate(SequenceStep.Phoneme(letter, isArticulationActive = true))
                    playbackManager.play("ph_$letter")
                    delay(step.durationMs + 500L) // 500ms pause between models
                }
                is Step.PlayKeyword -> {
                    onStepUpdate(SequenceStep.Keyword(step.word))
                    playbackManager.play("kw_${letter}_${step.word}")
                    delay(step.durationMs)
                }
            }
        }
    }
}
```

3.2 Speech Recognition Subsystem & Visualizer (FR-03, NFR-ASR-01, NFR-PERF-01)

To eliminate child hesitation at the mic and enable real-time visualization, the standard Vosk `SpeechService` wrapper is to be replaced with a low-level, non-blocking `AudioRecord` pipeline (planned for after Round 2; the shipped mic shows its four states with a time-based ripple, §2.0).

3.2.1 AudioRecord Loop & RMS Amplitude Pipeline

```
[Device Microphone (16 kHz Mono PCM)]
                |
                v
       [AudioRecord Buffer] (1024 samples / 64 ms chunk)
                |
       +--------+---------+
       |                  |
       v                  v
[Compute RMS Amplitude]  [Vosk Recognizer.acceptWaveForm()]
       |                  |
       v (0.0 to 1.0)     v (Partial/Final Result)
[MicStateVisualizer]     [SayItJudge Decision Logic]
```

- **Audio Settings:** Sample Rate: 16,000 Hz, Channel: `CHANNEL_IN_MONO`, Encoding: `ENCODING_PCM_16BIT`.

- **RMS Calculation:** For each 1024-sample buffer, root-mean-square amplitude is calculated:

$$\text{RMS} = \sqrt{(1/\text{N}\ \Sigma_{\text{i=1}}{}^{\text{N}}\ \text{x}_{\text{i}}{}^{2})}, \qquad \text{Normalized Level} = \text{coerceIn}((20\ \log_{10}(\text{RMS} + 10^{-5}) - \text{Floor})\ /\ \text{Range}, 0.0, 1.0)$$

- **Tap-to-Listening Transition:** When the user taps the mic button (or carrier finishes), the ViewModel immediately mutates UI state to `MicState.LISTENING` in ≤ 20 ms, while the background recording coroutine initializes.

3.2.2 MicStateVisualizer Composable (planned)

```
@Composable
fun MicStateVisualizer(
    state: MicState,
    audioLevel: Float, // 0.0 to 1.0
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    val animatedScale by animateFloatAsState(
        targetValue = if (state == MicState.LISTENING) 1.0f + (audioLevel * 0.45f) else 1.0f,
        animationSpec = spring(stiffness = Spring.StiffnessLow)
    )
    val rippleAlpha by animateFloatAsState(
        targetValue = if (state == MicState.LISTENING) 0.35f + (audioLevel * 0.5f) else 0.0f
    )

    Box(contentAlignment = Alignment.Center, modifier = modifier.size(100.dp)) {
        if (state == MicState.LISTENING) {
            Box(
                Modifier
                    .size(96.dp)
                    .scale(animatedScale)
                    .background(Color(0xFFFBBF24).copy(alpha = rippleAlpha), CircleShape)
            )
        }
        IconButton(
            onClick = onClick,
            modifier = Modifier.size(72.dp).background(state.backgroundColor, CircleShape)
        ) {
            Icon(state.icon, contentDescription = state.description, tint = Color.White)
        }
    }
}
```

3.3 Tutoring Layer & State Machine (FR-03, NFR-ASR-01, FR-NEW-REC)

3.3.1 SayItJudge with Dynamic Letter Grammars

Vosk is instantiated with a constrained per-letter runtime grammar rather than open vocabulary:

```
{
  "letter": "m",
  "mode": "WORD",
  "grammar": ["mouse", "m", "em", "ma", "muh", "[unk]"],
  "targetToken": "mouse",
  "foils": {
    "m": "LETTER_NAME",
    "em": "LETTER_NAME",
    "ma": "ADDED_VOWEL",
    "muh": "ADDED_VOWEL"
  }
}
```

- **Error Classification:**
  - **Scoring mode (hybrid):** the scored check is word mode (the key word, e.g. "mouse"), as in `SpeechValidator.grammarFor()` today. Pure-sound grammars (`<target_m>`) are used for the recall check only if the on-device Vosk test with children passes; the Week 4 spike found that Vosk cannot confirm a held /m/, so `SpeechValidator.SOUND_MODE_ENABLED = false` (`docs/spikes/vosk-foil-spike.md`).
  - **Confidence:** no fixed threshold is set in this design. If a confidence threshold is introduced, it is tuned on Round 2 data, validated on a held-out set, and kept in configuration (refactor spec §6.2, §6.4).
  - Utterance matches `targetToken` → `JudgeResult.CORRECT`.
  - Utterance matches `"em"` → `JudgeResult.ERROR(ErrorType.LETTER_NAME)`.
  - Utterance matches `"ma"` → `JudgeResult.ERROR(ErrorType.ADDED_VOWEL)`.
  - Utterance matches `"[unk]"` or nothing → `JudgeResult.ERROR(ErrorType.UNKNOWN)`.

3.3.2 TutorPolicy Finite State Machine (FSM)



3.4 Decodable Blend It Word Bank Refactoring (FR-13)

The seeded `BlendItWord` list (`BLEND_IT_WORD_SEEDS` in `di/DatabaseModule.kt`) carries the replacements below and is written on every app start, so existing installs receive them; each replacement keeps the word id it replaces, so learner progress is unaffected (card 15, implemented 2026-10-09; teacher confirmation of the list pending). The 5 invalid words identified during MVP validation are replaced:

Chapter	Unlocked Letters	Purged Word	Violation Reason	Replacement Word	Valid Letters Used

Chapter	Unlocked Letters	Purged Word	Violation Reason	Replacement Word	Valid Letters Used
Ch 1	m, s, a, i	**AIM**	Contains vowel team *ai*	**AM**	a, m
Ch 2	+ o, b, e, u	**BEE**	Contains vowel team *ee*	**SUM**	s, u, m
Ch 3	+ t, k, l, y	**TOY**	Contains diphthong *oy*	**TUB**	t, u, b
Ch 3	+ t, k, l, y	**BOY**	Contains diphthong *oy*	**YAM**	y, a, m
Ch 7	+ q, v, x, z	**ZOO**	Contains vowel team *oo*	**ZIP**	z, i, p
Ch 7	+ q, v, x, z	**QUIZ**	*qu* = /kw/ digraph	**QUIZ**	Kept as the documented exception (named in the seed comment and excluded by `BlendItWordSeedsTest`)

3.5 Database Architecture (Room Schema v4, planned) & Telemetry Logger (FR-14, FR-NEW-TEL)

3.5.1 Room Database Entities

```
// Profile Entity: Multi-Profile Management (FR-14).
// EXISTS in schema v3 (data/local/entity/ProfileEntity.kt); unchanged in v4.
// The name is optional for the child: onboarding is avatar-only and the default
// name is the avatar's name; a parent can rename the profile in the Parent Zone.
@Entity(tableName = "profiles")
data class ProfileEntity(
    @PrimaryKey(autoGenerate = true) val profileId: Long = 0,
    val name: String,
    val avatarResId: Int,
    val totalStars: Int = 0,
    val currentStreak: Int = 0,
    val lastPlayedAt: Long = System.currentTimeMillis(),
    val createdAt: Long = System.currentTimeMillis()
)

// PLANNED for schema v4: LetterProgressEntity and TelemetryEventEntity below.

// Letter Progress Entity: Spaced Retrieval Mastery (FR-NEW-REC)
@Entity(
    tableName = "letter_progress",
    foreignKeys = [
        ForeignKey(
            entity = ProfileEntity::class,
            parentColumns = ["profileId"],
            childColumns = ["profileId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index(value = ["profileId", "letter"], unique = true)]
)
data class LetterProgressEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val profileId: Long,
    val letter: String,
    val masteryState: String, // "NEW", "INTRODUCED", "NEEDS_PRACTICE", "SECURE"
    val echoPassCount: Int = 0,
    val recallPassCount: Int = 0,
```

```
    val lastSeenEpoch: Long = 0,
    val nextReviewEpoch: Long = 0
)

// Telemetry Event Entity: Diagnostic Interaction Logging (FR-NEW-TEL)
@Entity(
    tableName = "telemetry_events",
    foreignKeys = [
        ForeignKey(
            entity = ProfileEntity::class,
            parentColumns = ["profileId"],
            childColumns = ["profileId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index("profileId"), Index("eventType")]
)
data class TelemetryEventEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val profileId: Long,
    val sessionId: String,
    val chapter: Int,
    val letter: String,
    val module: String, // "HEAR", "SAY", "FIND", "BLEND"
    val eventType: String, // "mic_tap", "speech_end", "asr_result", "feedback_shown", etc.
    val elapsedRealtimeMs: Long, // Monotonic SystemClock.elapsedRealtime() for latency
    val wallClockEpoch: Long, // Epoch timestamp for audit correlation
    val resultValue: String?, // Recognized text or correct/incorrect
    val asrConfidence: Float?,
    val errorType: String? // "LETTER_NAME", "ADDED_VOWEL", "SUBSTITUTION", "UNKNOWN"
)
```

3.5.2 Local CSV Export Manager

A local exporter reads `telemetry_events` for the active profile, formats records into RFC 4180 CSV, writes the file to `context.filesDir/exports/`, and launches the native Android `Intent.ACTION_SEND` share sheet behind the arithmetic gate.

3.5.3 System Entity-Relationship Diagram

Figure 1 shows the Room database `playit_db` as built (schema version 3, `PlayItDatabase.kt`).

- **Tables:** 11 tables in three groups:
  - **Curriculum, seeded on every open:** `phonemes`, `letter_groups`, `letter_group_members`, `blend_it_words`.
  - **Learners:** `profiles`, `achievements`.
  - **Progress and attempts:** `lesson_progress`, `blend_it_progress`, `say_it_attempts`, `find_it_attempts`, `blend_it_attempts`.
- **Foreign keys:** every one is `ON DELETE CASCADE`, so deleting a profile removes all of that child's rows (FR-14).
- **Star totals:** the stored `profiles.totalStars` column is not what the app shows. The map and profile cards show `SUM(lesson_progress.starsEarned) + SUM(blend_it_progress.starsEarned)`, computed in `ProfileDao` (card 28).
- **Planned tables:** the two schema v4 tables of §3.5.1 (`letter_progress`, `telemetry_events`) are grey and dashed.

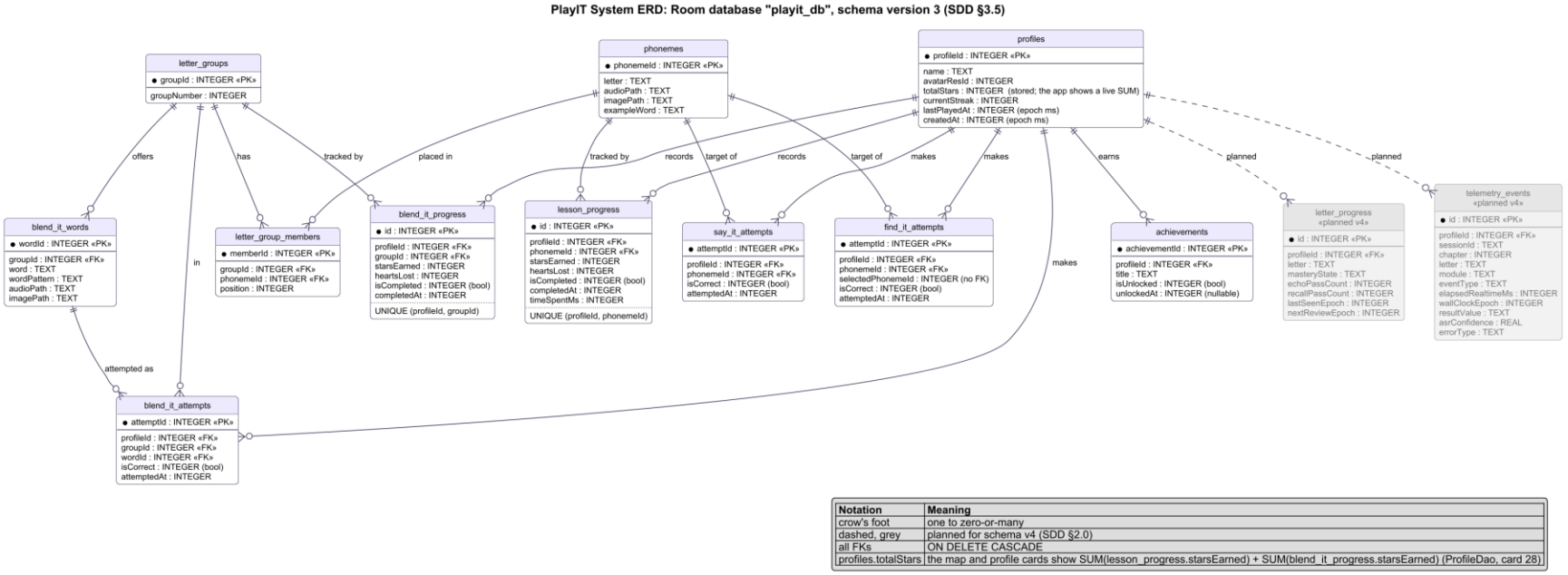


Figure 1. *System ERD: Room database playit_db, schema version 3, with the planned v4 tables in grey*

3.6 Pediatric Design System Tokens & Accessibility (NFR-ACC-01, NFR-ACC-02)

3.6.1 Touch Targets and Adaptive Dimensions (implemented)

The 64 dp touch floor and the 16 sp text floor are carried by design tokens rather than one global modifier. `windowProfileFor(widthDp, heightDp)` picks `COMPACT` (under 700 dp tall), `REGULAR` or `WIDE` (600 dp and wider), and `LocalPlayItDimens` provides the matching `PlayItDimens` (letter card height, primary button, mic size, Find It card size, tile size, map node size). Every child-facing control uses `heightIn(min = 64.dp)` or a token at least that large. `LessonScaffold` pins the top bar, header and bottom bar, and lets the body fit or scroll, capped at 560 dp wide on tablets:

```
@Composable
fun LessonScaffold(
    topBar: @Composable () -> Unit,
    header: @Composable () -> Unit,
    bottomBar: @Composable () -> Unit,
    modifier: Modifier = Modifier,
    content: @Composable ColumnScope.() -> Unit,
)
```

`LayoutMatrixTest` renders Hear It, Say It, Find It, Blend It, the map, the complete screens, splash and the name prompt at 360x640, 360x740, 411x891 and 800x1280 dp (and at font scale 1.3), and fails if the main action is below the window.

3.6.2 Articulation Cue and Sound Captions (implemented; pictures pending)

Each letter maps to one of 9 mouth-shape groups (`ArticulationGroup`, pure Kotlin), an approximation for a child that a teacher checks during Gate 3. Captions come from `CaptionText.forClip(path, letter, word)`: a phoneme clip shows `/m/`, a key word shows the word, a carrier shows its line, and a pause shows nothing. `AudioPlayer.playSequence(paths, onItemStart, onComplete)` reports each clip as it starts, so Hear It captions the modeling sequence live.

```
@Composable
fun ArticulationCue(group: ArticulationGroup, size: Dp, modifier: Modifier = Modifier)

@Composable
fun CaptionBubble(caption: String?, modifier: Modifier = Modifier)
```

`ArticulationCue` draws nothing until its picture is in `assets/images/mouth/` (image release of card 25). Hear It shows a 72 dp cue under the letter card (no caption in the child view since card 28); Say It shows a 96 dp cue beside the mic from the second miss ("Watch my lips", spec Table 7).

3.7 Module Class Diagrams

Figures 2 to 7 give one class diagram for each functional module of SRS §3.2.

- **Source:** traced from the code on `refactor/hear-say-it` (checked 2026-10-11).
- **Layout:** each diagram is grouped by the layers of §2.2 (presentation, domain, data).
- **Planned classes:** classes the earlier sections describe as planned are grey and marked «planned», with a dashed line to the class they would extend or replace (for example `AudioComposer`, `LearnerModel`, `TelemetryLogger`). Everything else exists in the app today.
- **Sources:** the diagram sources are in `docs/diagrams/src/`.

3.7.1 Module 1: Letter-Sound Acquisition (Hear It)

`HearItViewModel` builds the modeling sequence with `HearItSequenceBuilder` and plays it through `AudioPlayer`. `ArticulationCue` reads its picture from the letter's `ArticulationGroup`.

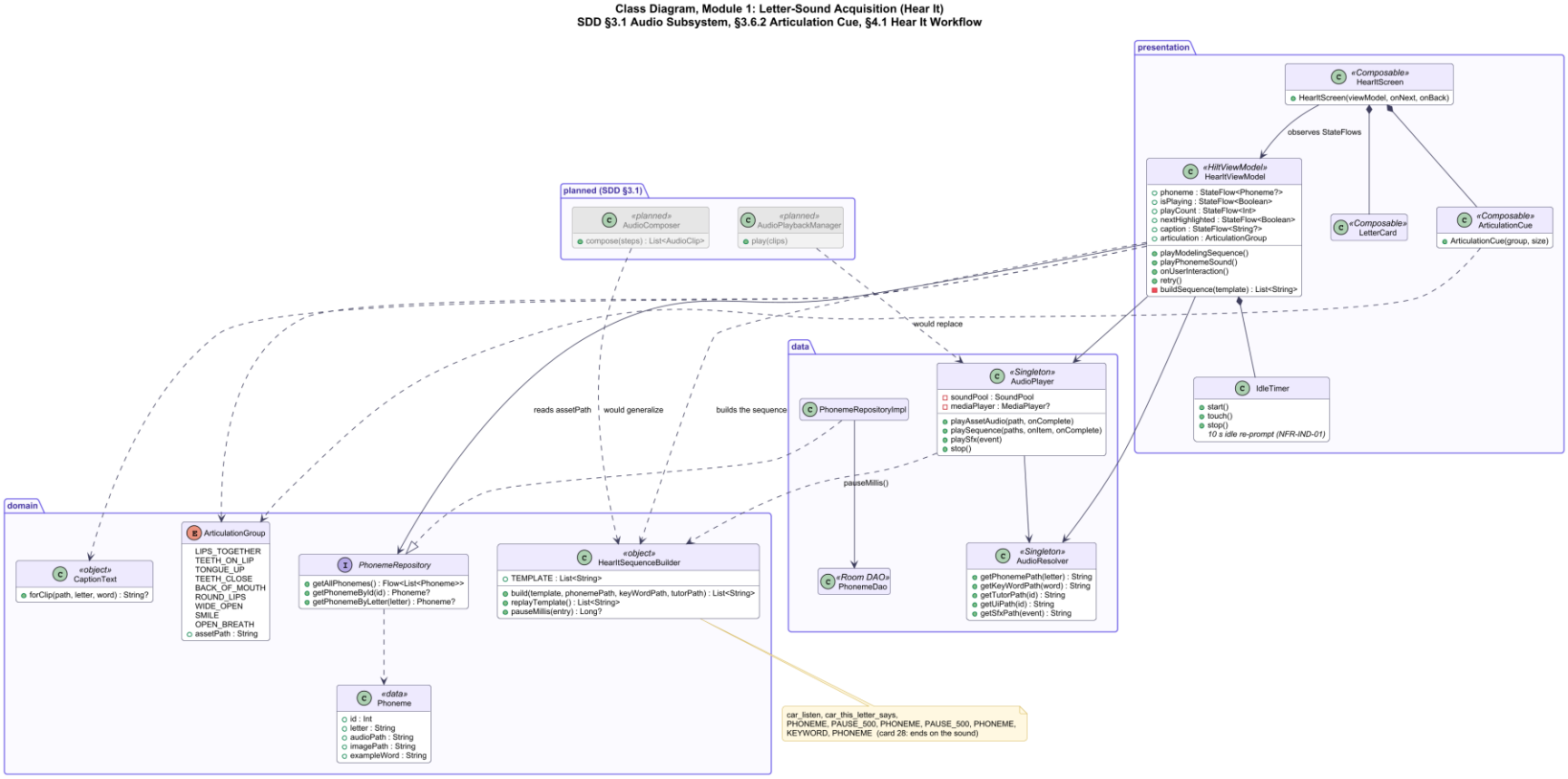


Figure 2. *Class diagram, Module 1: Hear It*

3.7.2 Module 2: Speech Production and Formative Tutoring (Say It)

- **Judging and the prompt ladder:** `SpeechValidator` (in the role of `SayItJudge`) judges the transcript. The stateless `TutorPolicy` maps the attempt number and the judgement to `Praise`, `Correct` or `LeadAndMoveOn`.
- **Mic state:** `MicStatus` drives the mic button.

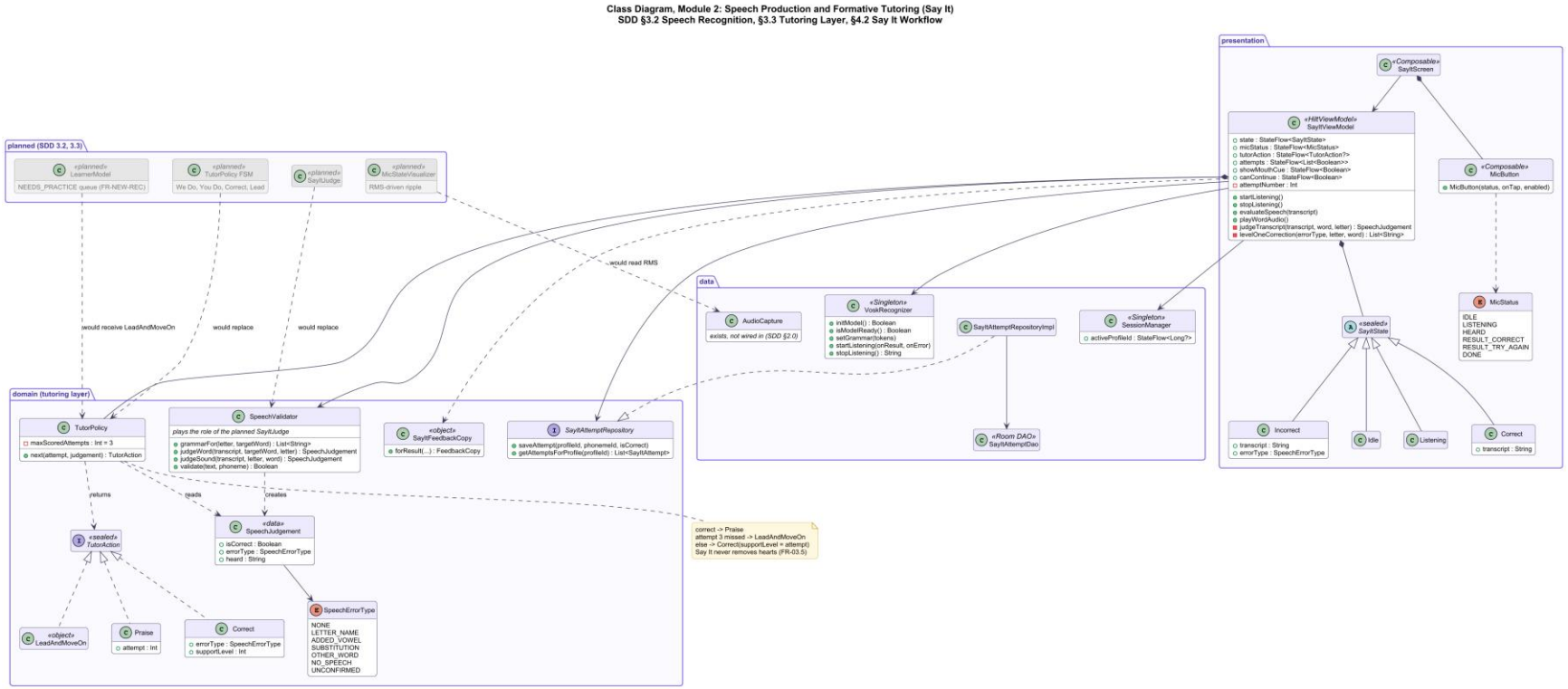


Figure 3. *Class diagram, Module 2: Say It*

3.7.3 Module 3: Phoneme Discrimination (Find It)

`GridGenerator` builds 5 pictures (3 targets, 2 distractors). `HeartManager` deducts hearts, recovers them and restarts with 3.

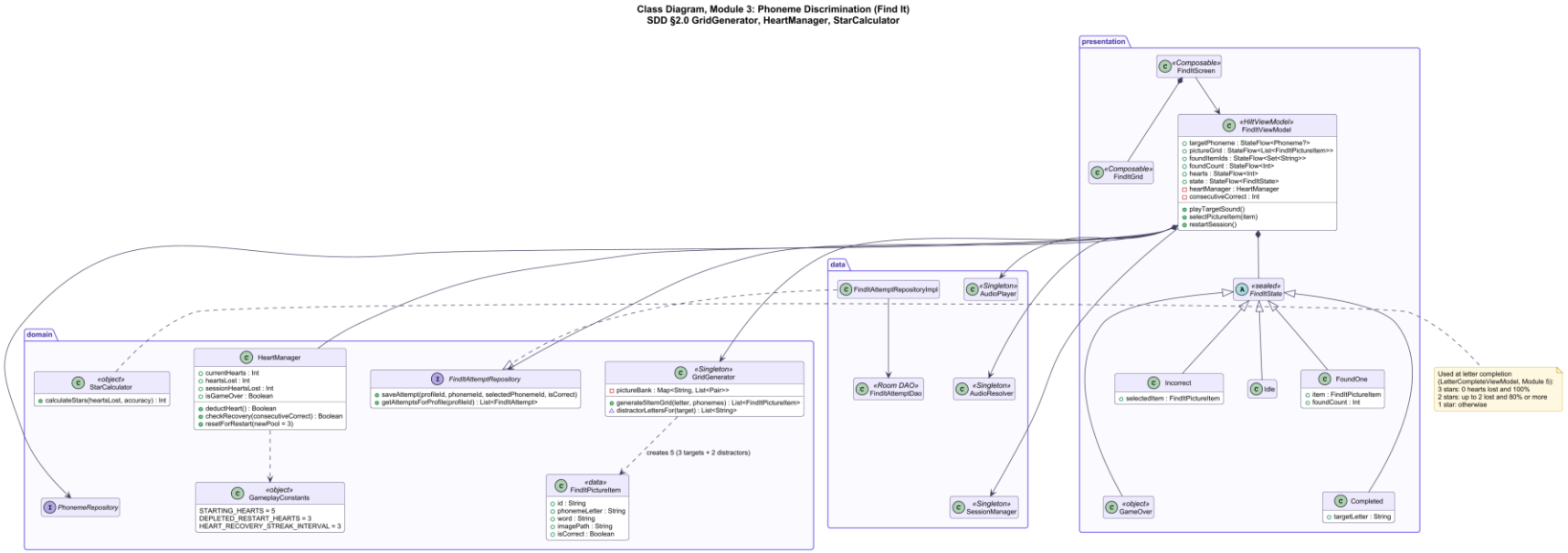


Figure 4. *Class diagram, Module 3: Find It*

3.7.4 Module 4: Word Construction Checkpoint (Blend It)

`BlendItViewModel` checks the tiles and runs the sound-out. `BlendItCompleteViewModel` scores the group with `BlendItStarThresholds` and saves `BlendItProgress`.

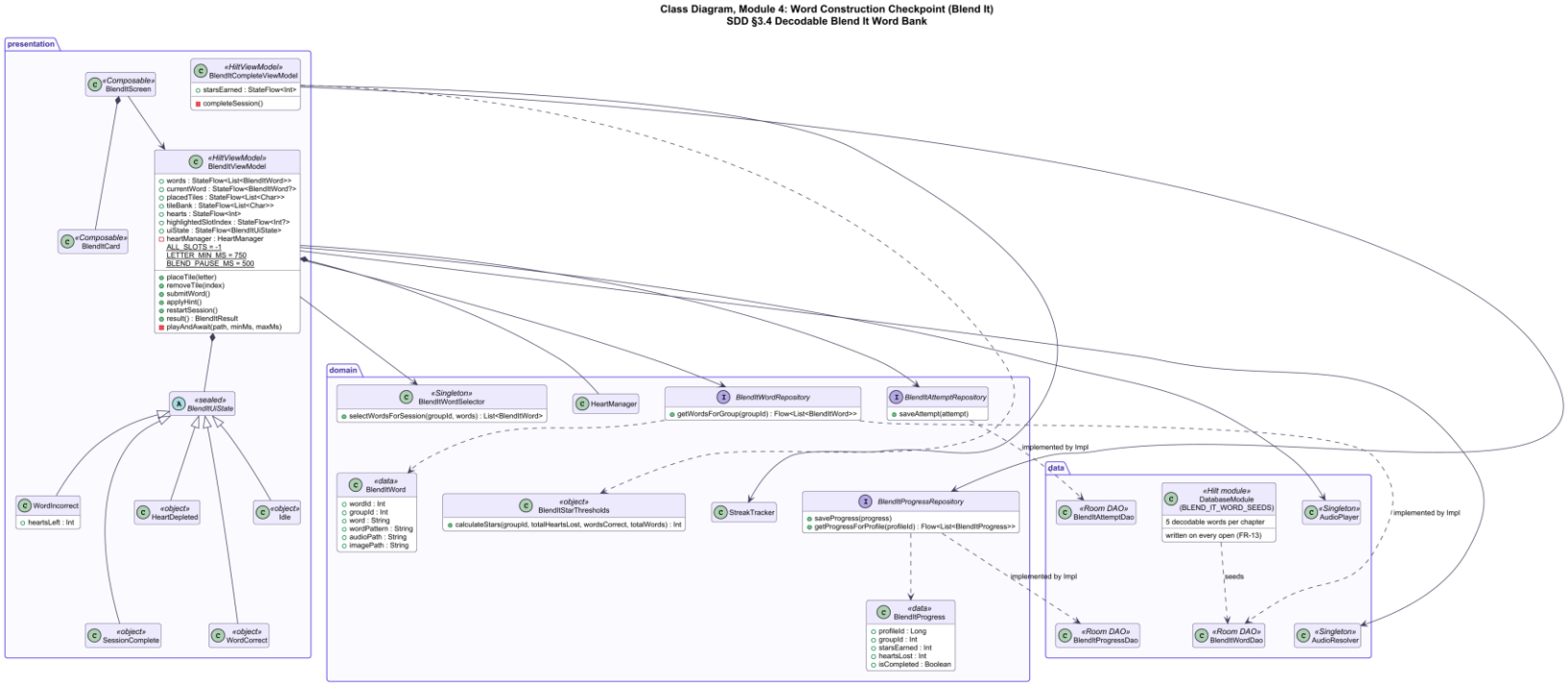


Figure 5. *Class diagram, Module 4: Blend It*

3.7.5 Module 5: Mastery Progression, Gamification and Multi-Profile

`SessionManager` holds the active profile. `LetterCompleteViewModel` saves stars and the streak. `MapViewModel` derives the nodes with `UnlockManager` and the live star total from `ProfileDao`.

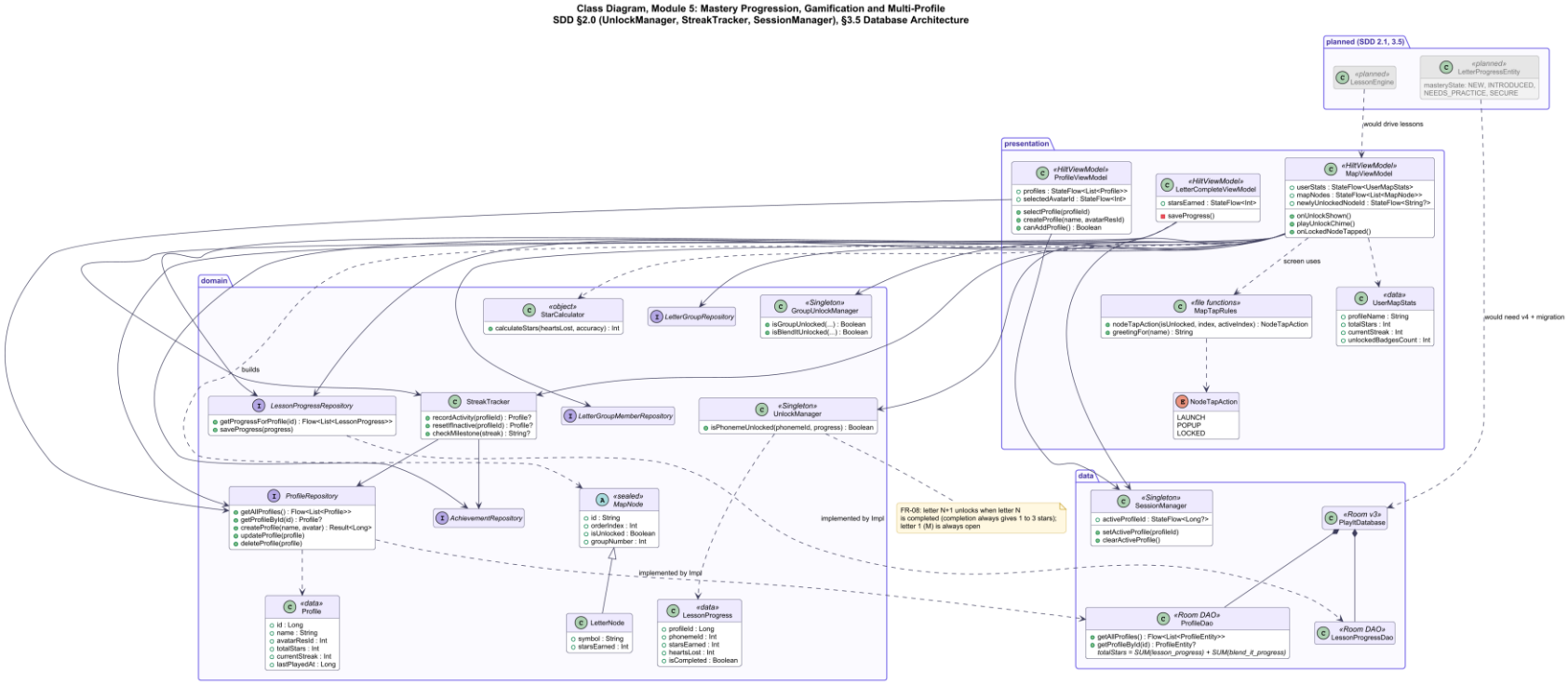


Figure 6. *Class diagram, Module 5: Progression, gamification and profiles*

3.7.6 Module 6: Telemetry, Parent Dashboard and Diagnostics

`ReportGenerator` aggregates the repositories into `ProfileDashboardData` and `ReportData`. `PdfExporter` writes the report. The arithmetic gate guards entry. The telemetry classes are planned.

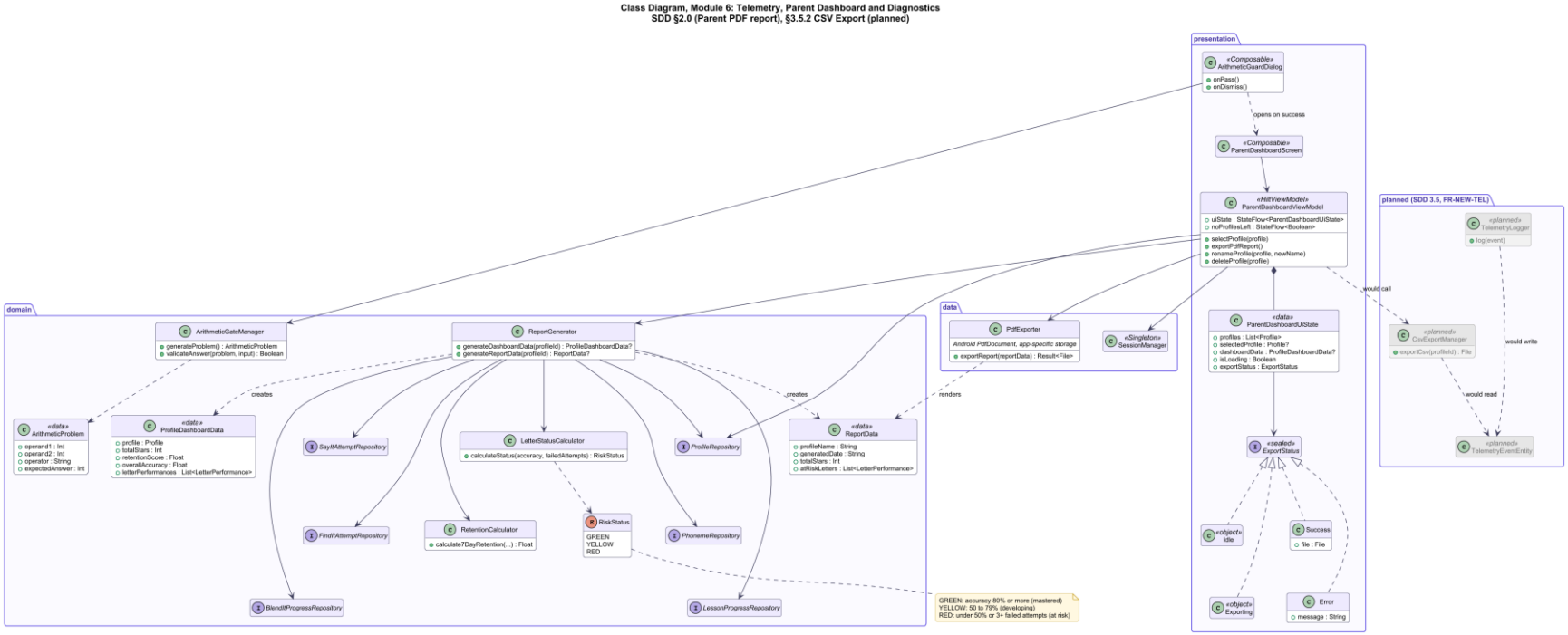


Figure 7. *Class diagram, Module 6: Parent dashboard and diagnostics*

4. Detailed Interface & Interaction Workflows

4.1 Hear It Modeling Sequence Workflow

```
User selects Letter -> Screen Loads -> SoundPool Preloads Letter Clips
  -> Step 1: Mascot plays "Listen!" carrier (Audio + Animation)
  -> Step 2: Letter card flips, displaying embedded Picture Mnemonic
  -> Step 3: Mascot plays "This letter says..." carrier
  -> Step 4: Pure phoneme plays 3 times (800ms hold or <=250ms stop)
             accompanied by ArticulationCue highlight and caption
  -> Step 5: Mascot plays Keyword audio ("mouse") + Keyword image
  -> Step 6: Pure phoneme plays 1 time
  -> Step 7: "Great listening! Tap the big button." -> Next pulses -> Say It
     (card 28: Hear It no longer ends with "Say it with me!")
```

Figure 8 shows the same workflow as the code runs it. Figures 8 to 13 give one sequence diagram for each module.

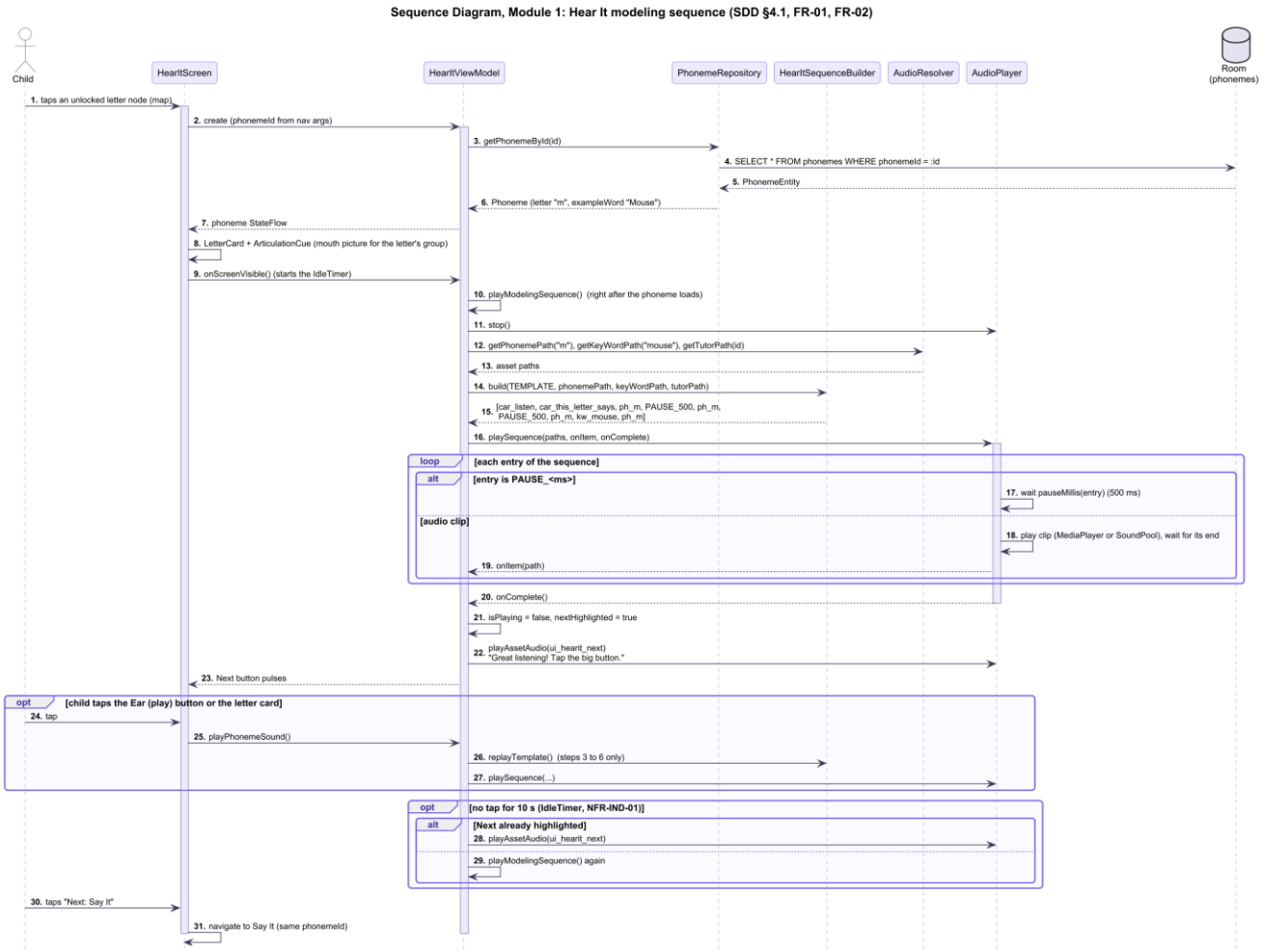


Figure 8. *Sequence diagram, Module 1: Hear It modeling sequence*

4.2 Say It Interaction & Correction Workflow

```
Screen Enters -> Mascot leads 2 Choral Turns ("Say it with me!")
  -> Mascot plays "Your turn!"
  -> MicState changes to LISTENING (<= 100ms); the ripple animates
     (time-based today; RMS-driven once the AudioRecord loop lands)
  -> Child vocalizes -> Vosk reports speech -> MicState changes to HEARD
  -> Vosk signals End-of-Speech
  -> SayItJudge (SpeechValidator today) checks the result against the per-letter grammar:
       [Branch A: Target Match]
         -> MicState changes to RESULT (Green)
         -> Success chime plays + specific praise ("Yes! /m/, lips together!")
         -> Award Star -> Advance to Find It
       [Branch B: Foil Match or Silence (Attempt < 3)]
         -> MicState changes to RESULT (Amber)
         -> Formative prompt plays (e.g. "Almost! Just /m/, no 'ah'")
         -> Lip cue enlarges -> Mic reactivates for retry
       [Branch C: Foil Match or Silence (Attempt == 3)]
         -> Mascot leads together-practice ("Let's say it together: /m/")
         -> LetterProgressEntity marked NEEDS_PRACTICE (planned: Room v4)
         -> Advance to Find It (Zero hearts deducted)
```

In the code today, the third miss saves the attempt rows and moves on; the `NEEDS_PRACTICE` mark waits for Room schema v4 (§2.0).

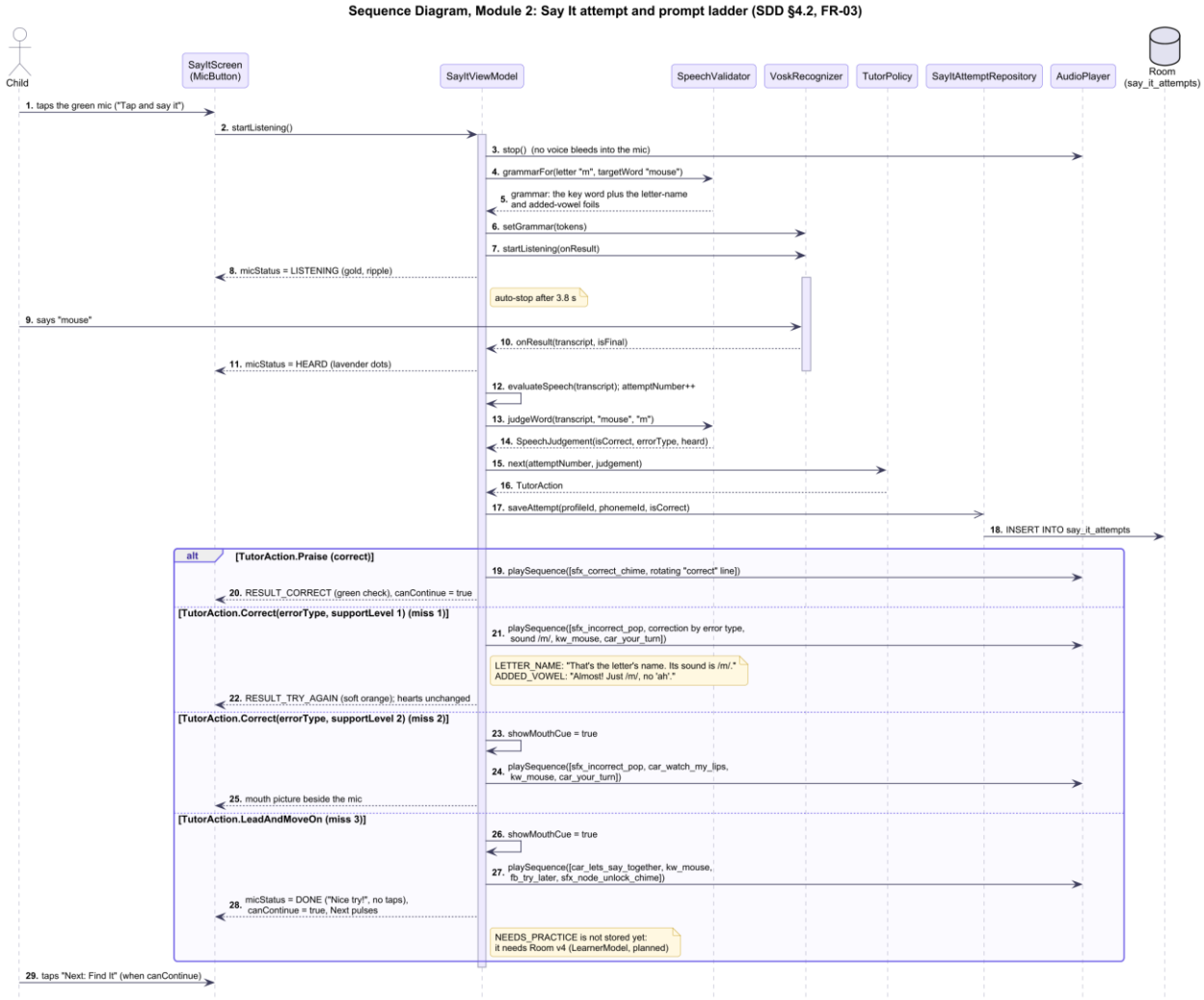


Figure 9. *Sequence diagram, Module 2: Say It attempt and prompt ladder*

4.3 Find It Tap, Hearts and Restart Workflow

- **A correct tap:** it counts towards the 3 targets. Every 3 correct taps in a row recover a heart, up to the starting pool.
- **A wrong tap:** it removes one heart. At 0 hearts the activity restarts with a 3-heart buffer (FR-05).
- **Letter completion:** the hearts lost in the session go to letter completion (§4.5).

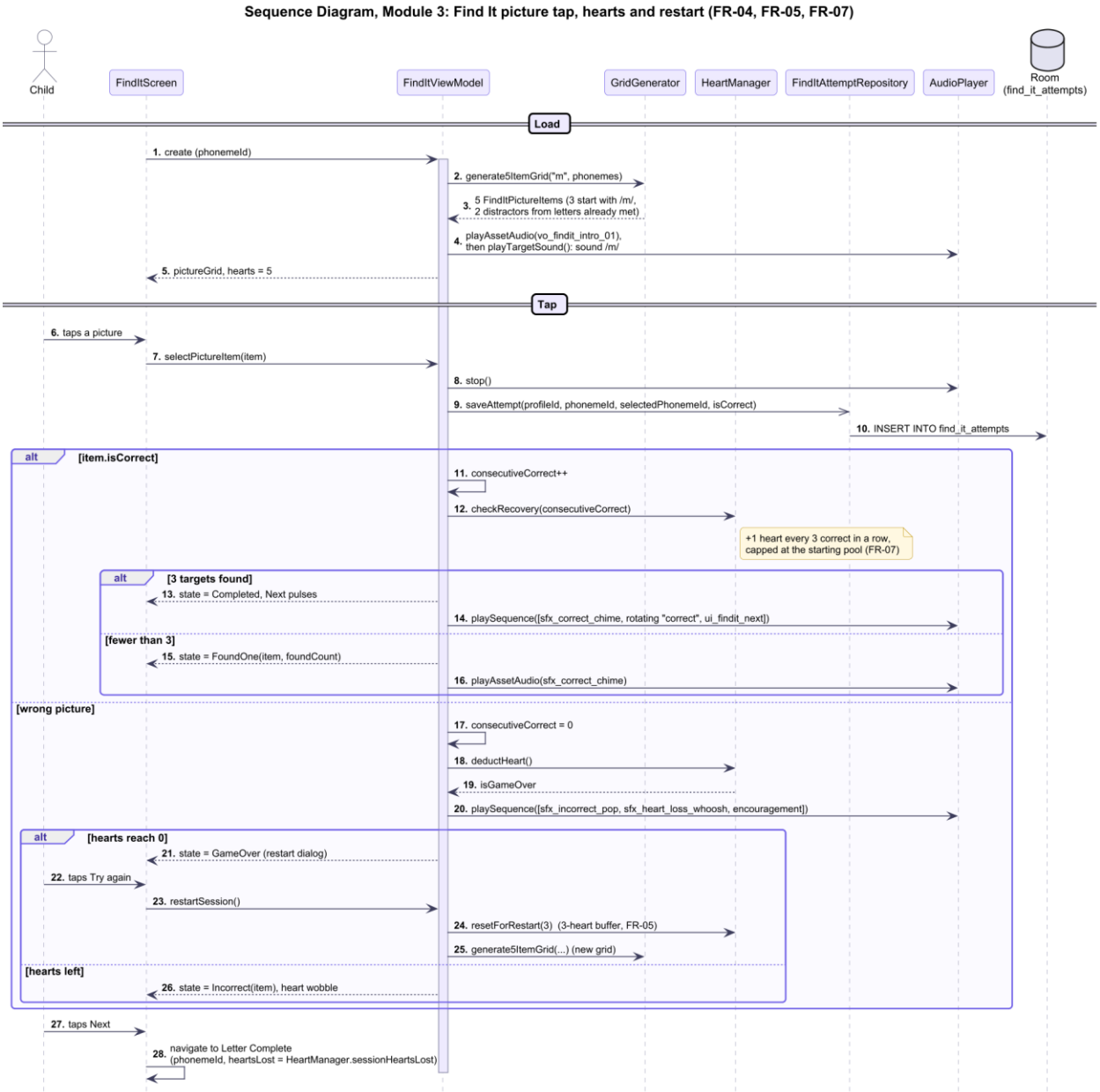


Figure 10. *Sequence diagram, Module 3: Find It*

4.4 Blend It Word and Group Completion Workflow

- **Checking a word:** each answer is saved.
- **A correct word:** the tiles light one by one while each letter sound plays to its end (at least 750 ms each). Then every tile lights together, and the whole word plays.
- **Group completion:** a finished group is scored and saved to `blend_it_progress`.

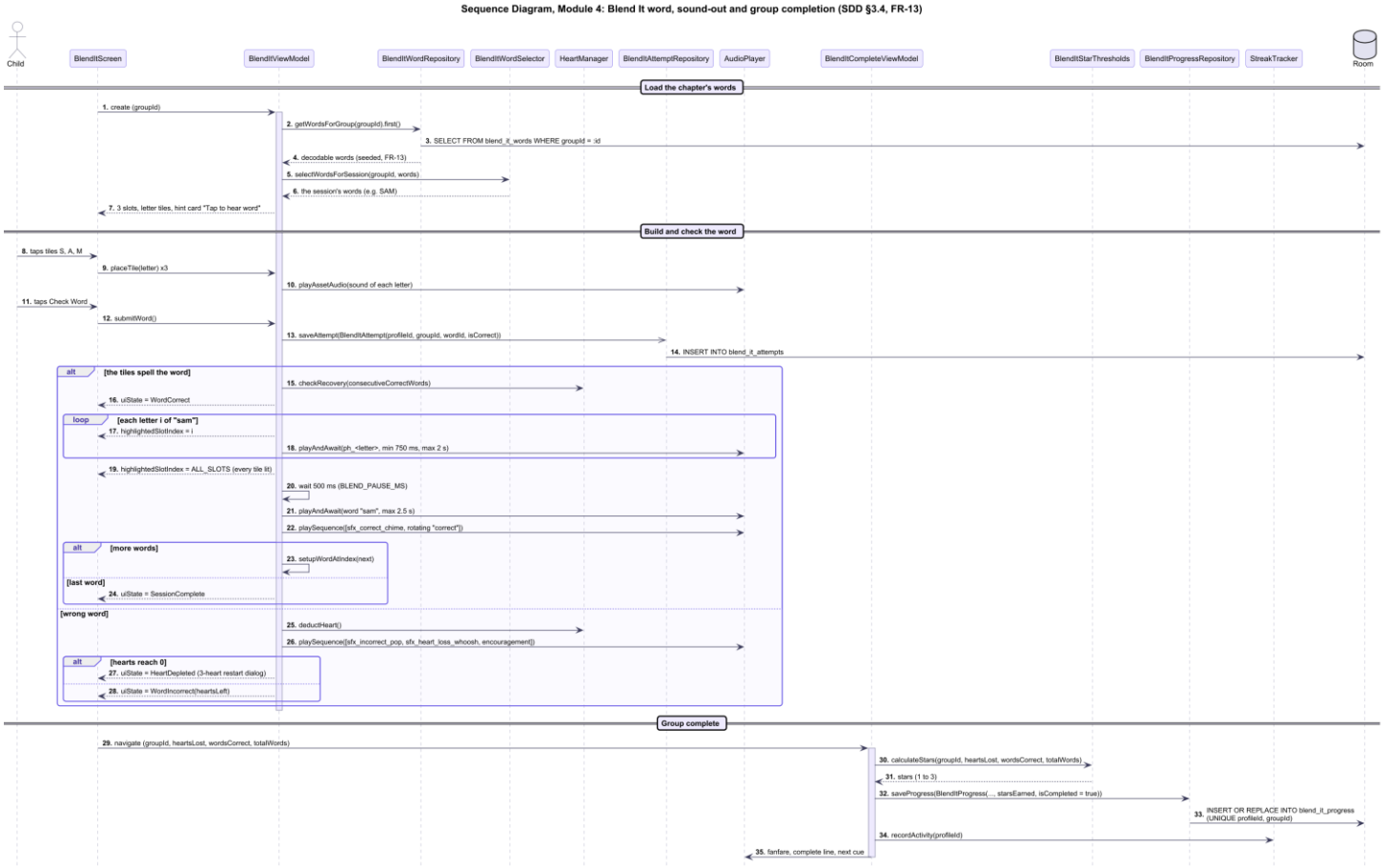


Figure 11. *Sequence diagram, Module 4: Blend It*

4.5 Profile, Stars, Streak and Unlock Workflow

- **Profiles:** the active profile scopes every read and write (FR-14).
- **Letter completion:** completing a letter saves its stars (FR-06) and updates the daily streak (FR-09).
- **Map update:** the map's Room flows re-emit, `UnlockManager` opens the next letter (FR-08), and the star total is recomputed in SQL.

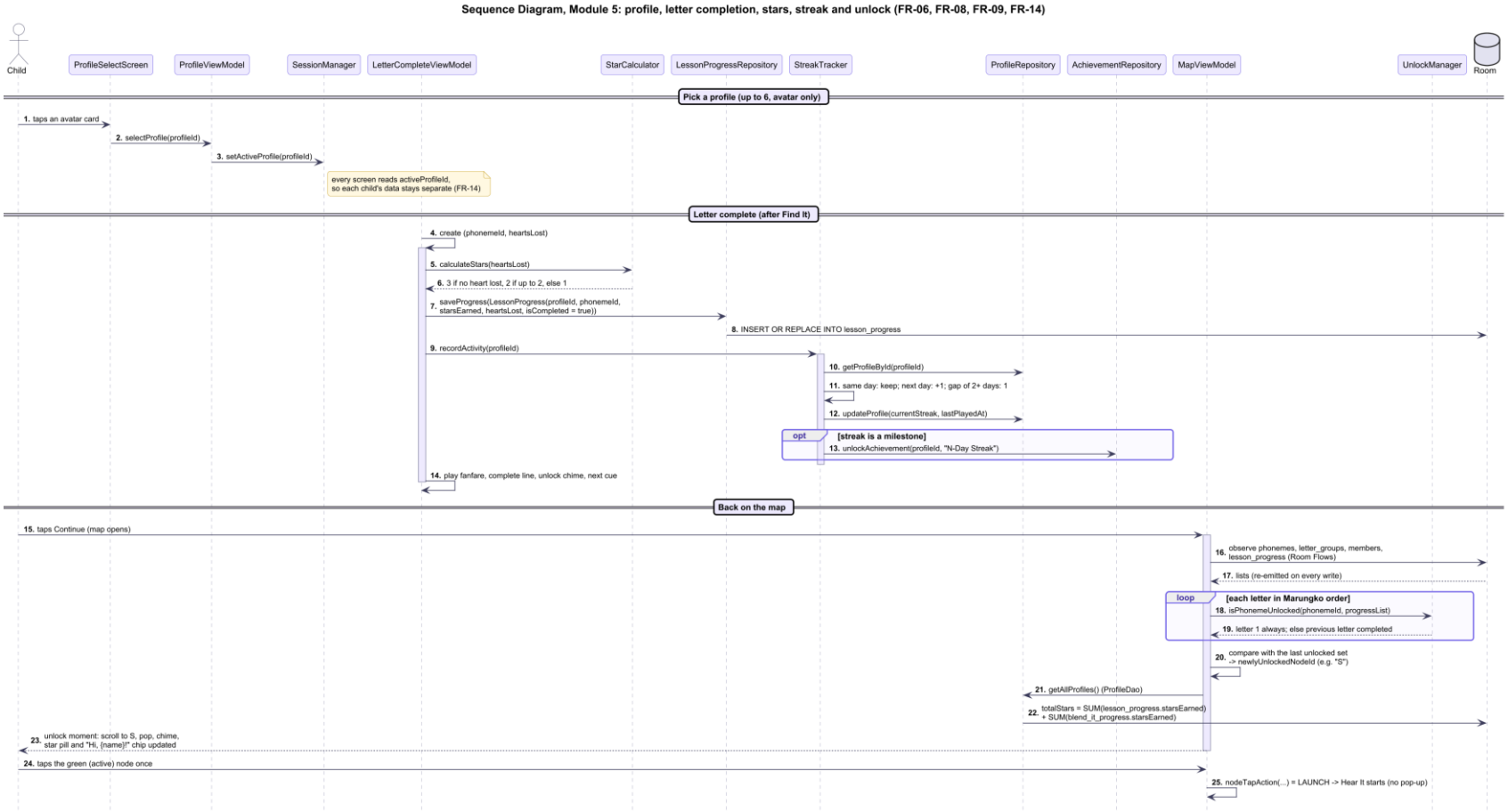


Figure 12. *Sequence diagram, Module 5: progression and profiles*

4.6 Parent Zone, Dashboard and Report Workflow

- **Entry:** an arithmetic problem guards the Parent Zone.
- **Dashboard and report:** the dashboard and the PDF report come from `ReportGenerator`.
- **CSV export:** the telemetry CSV export of §3.5.2 is shown in grey because it is planned.

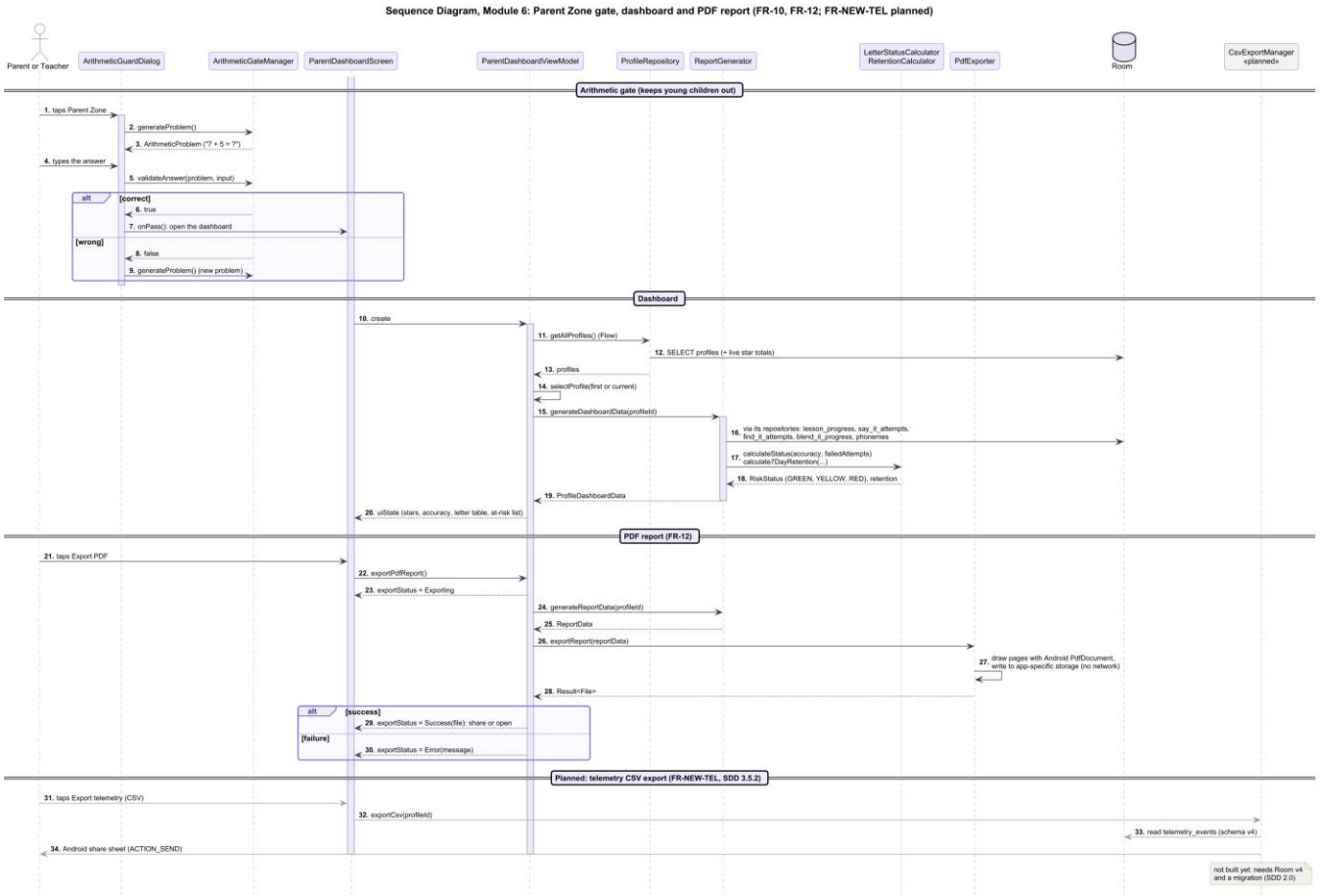


Figure 13. *Sequence diagram, Module 6: Parent dashboard and report*

5. Technology Stack & External Dependencies

Layer / Subsystem	Technology / Library	Version	Technical Justification & Usage Scope
Language	Kotlin	1.9.22	100% pure Kotlin domain layer with coroutines and Flow.
UI Framework	Jetpack Compose	1.5.4	Declarative reactive UI toolkit utilizing Material Design 3 tokens.
Speech Engine	Vosk Android SDK	0.3.47	Lightweight offline speech recognition (`vosk-model-small-en-us-0.15`).
Audio I/O	Android AudioRecord / SoundPool	Native	Non-blocking 16kHz PCM recording and zero-latency SoundPool playback.
Local Database	Android Jetpack Room	2.6.1	SQLite ORM managing profiles, spaced retrieval states, and telemetry.
Document Export	Android PdfDocument	Native	Zero-dependency local PDF progress report generation.
Audio Synthesis	Kokoro-82M / Chatterbox-Turbo	Apache-2.0 / MIT	Offline neural TTS for carrier phrases and cloned held phonemes (build-time tools, not shipped in the app). Short vowels: a team member's recordings voice-converted with ElevenLabs (**[proposed]**, adviser confirmation pending).

6. Verification & Architectural Testing Plan

1. **Audio Latency & Purity Test Suite (`TC-AUD-01`, `TC-AUD-02`):** Automated instrumentation tests verifying SoundPool stream start latency ≤ 50 ms and gate verification integrity.
2. **Microphone Reactive Visualizer Suite (`TC-MIC-01`, `TC-MIC-02`):** Robolectric/Espresso tests measuring state transition timestamp deltas (≤ 100 ms) and ripple scaling proportional to mock PCM input.
3. **ASR Discrimination Suite (`TC-ASR-01` to `06`):** Audio replay harness feeding benchmark child recordings (normal, foil, noise) into Vosk to verify ≥ 80% agreement and ≤ 15% false reject limits.
4. **Room Schema Migration Suite (`TC-DB-01`, `TC-DB-02`):** `MigrationTestHelper` verifying upgrade from Schema v3 (current) to Schema v4 without data loss. Prerequisites: set `exportSchema = true` and replace `fallbackToDestructiveMigration()` with explicit migrations (both planned).